

LES SESSIONS SOUS PHP 4

Les sessions (apparues avec PHP4) qui nous permettent de faire passer nos variables à travers les différentes pages. Certes il y a d'autres façons tel que les querystrings (passage des variables par l'url) mais il est plus elegant et plus sécurisé d'utiliser les sessions. Les sessions sont adaptées à la sauvegarde de données confidentielles ou importantes. On peut citer quelques exemples courants de leur mise en application :

- authentifier un visiteur ;
- garder des informations sur un utilisateur tout au long de sa présence dans votre application ;
- gérer le panier d'achat d'un internaute sur votre site marchand ;
- mettre en place des formulaires en plusieurs parties et donc retenir les informations fournies dans les pages précédentes ;
- effectuer un cache par utilisateur de certaines actions coûteuses en ressources.

Qu'est-ce qu'une session ?

Au lieu de stocker vos informations chez le visiteur, vous les stockez sur le serveur. Techniquement, vous attribuez au visiteur un identifiant. À chaque fois qu'il revient en annonçant cet identifiant, PHP récupérera toutes les informations qu'il avait sauvegardé et qui sont relatives à ce visiteur.

Voyons voir maintenant comment fonctionnent ces sessions par un petit exemple concret :

Gestion d'un espace membres.

Sommaire
Formulaire d'identification
Script d'identification
Ouverture de la session
Enregistrement des variables
Vérification de l'identification du membre
Destruction de la session
Recapitulatif de toutes les fonctions

Sommaire

Pour que vous compreniez mieux le fonctionnement des sessions nous créerons un espace membre. Commençons par le principal : la table "membres" Nous ne créerons pas une multitude de champs dans notre cas, nous n'aurons besoin que de 3 champs : id, login et pass qui nous serviront à vérifier l'identité du membre.

```
CREATE TABLE membres ( // création de la table id
tinyint(4) NOT NULL auto_increment, // le champs id
qui sera indexé
login varchar(50) NOT NULL, // champs pour stocker le
login du membre
pass varchar(20) NOT NULL, // pour stocker le pass du
membre
UNIQUE KEY (id)
);
```

Le formulaire d'identification : formulaire.php

Jusque là tout va bien pour vous , c'est du déjà vu, le formulaire d'identification ne risque pas non plus de poser de problème vu que ce n'est que du html. Voici le code :

```
<table width="300" cellspacing="1" border="1"><form
method="post" action="identifier.php">
<tr><td bgcolor="darkblue"><center><font
color="white">Formulaire
d'identification</font></center></td></tr>
<tr><td><center><input type="text"
name="login"></center></td></tr>
<tr><td><center><input type="password"
name="pass"></center></td></tr>
<tr><td><center><input type="submit"
value="Identifier"></center></td></tr>
</form></table>
```

Voyons voir ce que ca donne en visuel

Formulaire d'identification : identifier.php

(N.B : le formulaire ci dessus n'est pas actif)

Passons à notre premier code : l'identification
Nos données concernant les membres sont stockées dans une table, il nous faudra donc nous connecter à la base, et selectionner les login et les pass et voir s'il y en a un qui concorde avec ceux qu'a entré le visiteur si oui on le redirige vers la zone membre et on ouvre la session, sinon on lui réaffiche le formulaire d'identification.

NB : Je vous conseille de lire les commentaires du code c'est là que tout se passe.

```
<?php
```

```
/******
* @Author : HbiLLo
* @Script : Zone Membre
* @Description : Ce script vous permet de gérer
* votre espace membre grâce à un système de
sessions.
* @Date : Jeudi 27 Juin 2002
* @Email : hbillo@az-php.com
* @WebTeam : Az-PHP
* @Team Website : http://www.az-php.com
* @Site Perso : http://www.phpcool.fr.st
*
******/
```

```
// ON inclut le fichier de configuration
require ("config.inc.php");
```

```
// On se connect à notre serveur Sql
mysql_connect($server,$user,$pass);
```

```
// On selectionne la base de données
mysql_select_db($base);
```

```
// On selectionne quand le champs login correspond au
login entré
```

```
// et le champs passe au pass entré.
$sql = "SELECT * FROM membres WHERE
login=$login and pass=$pass";

// On execute la requête de selection
$res = mysql_query($sql);

// On compte le nombre de ligne des resultats
// 1 : si valide 0 si aucun login ne correspond
$exist = mysql_numrows($res);

// Si la variable $exist = 0 --> login inexistant ou faux
pass
if(!$exist) {
    // On affiche ce message d'erreur
    echo "<center>Veuillez vérifier vos
données</center>";

    // On inclut le formulaire d'identification
    include("formulaire.php");
}

// Sinon, si le login et pass sont valides
else {

// On ouvre la session
session_start();

// On enregistre la variable login qu'on fera passer sur
toutes les pages
// ATTENTION : Notez bien l'absence de $ devant login
session_register("login");

// On affiche un message de bienvenu
echo "<center>Bienvenu sur votre espace membre :
$login</center>";

// Notons que la session a un nom par défaut et un
identifiant
// Nom par défaut : PHPSESSID
// Id Aleatoire généré lors de l'ouverture de la session

// définissons d'abord les variables
$sess_nom = session_name();
$sess_id = session_id();

// Affichons si on veut le nom est l'id de la session
echo "Le nom de la session est :<br> ";
echo $sess_nom." et l'id est : ".$sess_id."";

// On affiche maintenant un lien vers une page d'essai
// Notez qu'on ne passe pas les variables par l'url
echo "<a href='\"page.php\"'>Page Essai</a>";

// A la fin on offre au membre la possibilité de se
deconnecter
echo "<a href='\"deconnexion.php\"'>Deconnection</a>";
}

?>

Maintenant nous allons étudier de près cette
"page.php" et voir comment nous ferons pour vérifier
si le visiteur est identifié, et les actions à effectuer
selon le cas.
```

Comment vérifier si le visiteur est identifié

Vérifier si le visiteur est identifié , c'est tout simplement
vérifier si la variable \$login est déclarée (isset) la façon
la plus simple de procéder est celle ci :

Code du fichier page.php (1ère façon)

```
<?php

// On commence par inclure le fichier de configuration
require ("config.inc.php");

// On vérifie si la variable login est déclarée
// Si elle ne l'est pas on redirige vers le formulaire
if(!$login) {
    echo "<center>Veuillez vous identifier
d'abord</center>";
    include ("formulaire.php");
}

// Si la variable est déclarée on affiche ce message
else {
    echo "Bienvenu sur votre espace membre $login";
}

?>
```

Cependant, le code ci dessus contient une importante
faille de sécurité (j'espère que vous l'aviez deviné) car
n'importe quel visiteur peut entrer l'url avec la variable
login déclarée et de ce fait le script php voyant la
variable, n'y verra que du feu et les données de votre
site se trouveront menacées.

Url trompeuse :
<http://www.votresite.com/page.php?login=quelconque>

Corrigeons ce bug : Nous allons voir la **deuxième
methode** possible ou on utilisera la vérification des
sessions afin d'éviter la faille exposée ci dessus. Pour
celà on aura recours à la fonction : **if(!
session_is_registered(variable))**

```
<?php

/*****
* Voici la version non defectueuse
* du fichier page.php, c'est ce même
* code que vous devrez copier sur toutes
* les pages nécessitant un enregistrement.
*****/

// On inclut le fichier de configuration
require ("config.inc.php");

// On ouvre la session avant tout
session_start();

// On vérifie si la session "login" est enregistrée
// Si la session "login" n'est pas enregistrée
if(!session_is_registered(login)) {
    // On affiche ce message d'erreur
    echo "<center>Vous devez vous identifier pour accéder
à cette page</center>";

    // On affiche le formulaire d'identification
    include ("formulaire.php");
```

```

}

// sinon, la session "login" est enregistrée
else {
// On affiche le message de bienvenue
echo "<center>Bienvenu sur votre espace membre
$login</center>";
}

?>

Et voilà vous saurez maintenant comment vérifier que
votre visiteur est identifié, nous reste maintenant le
fichier deconnexion.php qui comme son nom l'indique
permettra au visiteur de se deconnecter (destruction de
la session).

<?php

// On commence par ouvrir la session
session_start();

// On désenregistre la session login
session_unregister("login");

// On supprime ttes les variables de la session
session_unset();

// On détruit totalement la session
session_destroy();

// On affiche le message de reussite
echo "Vous n'êtes plus connecté !";

// On réaffiche le formulaire d'identification
include ("formulaire.php");

?>

Pour configurer le fichier de configuration je pense
que vous n'aurez pas trop de mal :-) on voit ça à
chaque fois donc !

```

```

<?php

// On declare les variables de connection
$server = "localhost";
$user = "";
$pass = "";
$base = "az-php";

?>

```

Nous allons maintenant reprendre toutes les fonctions
 de sessions que nous avons vous et utilisées au cours
 de la création de notre script, afin de donner quelques
 précisions qui vous seront utiles plus tard.

Recapitulation des fonctions de sessions

```

?php

session_start();
/*****
* Cette fonction ouvre une session elle doit être utilisée
au tout
* début de votre code, sinon elle peut engendrer des
erreur du type
* ( headers already sent !)
* *****/

session_name();
/*****
* Ceci retourne le nom de votre session qui est tjrs par
défaut
* egal à PHPSESSID que vous verrez apparaître sur
vos liens lors de l'utilisation des sessions.
* *****/

session_id();
/*****
* C'est l'identifiant aléatoire crée lors de l'ouverture de
la
session *****/
session_register("variable");
/*****
* "Variable" est la variable que vous souhaitez
emmagasiner, il vous
* faudra enregistré chaque variable à passer, et notez
qu'on ne met
* pas de $ devant la variable à enregistrer
* *****/

session_is_registered("variable");
/*****
* Sert à vérifier que la session "variable" est bien
enregistrée
* et donc que la variable passe bien.
* utilisée pour vérifier si le visiteur est loggé ou pas.
* *****/

session_unregister("variable");
/*****
* Sert à désenregistret la session consacrée à
"variable", utilisée
* lorsqu'un utilisateur veut se deconnecter par
exemple.
* *****/

session_unset();
/*****
* Cette fonction désenregistre toutes les sessions en
cours.
* *****/

session_destroy();
/*****
* Cette fonctionne a pour but de détruire toutes les
sessions
* ouvertes et celles enregistrées.
* Utilisée aussi lors d'une deconnection.
* *****/

?>

```

EXEMPLE 2 DE CREATION DE SESSION

```
<? //on démarre la session
session_start();
```

```
//on affiche le nom et le numéro de session
print 'le nom de la session est '.session_name().' l'id
de session est '.session_id();
Jusque là, rien de très complexe, si vous ne voyez pas
de résultat, rafraîchissez la page, et vous pourrez lire
ceci:
le nom de la session est PHPSESSID l'id de session
est azd1258996adkjhs44hjdkus
```

Enregistrez un variable

Créer une page formulaire nommée form.php par exemple avec ce genre de code

```
<html>
<head></head>
<body>
<form method="post" action="enregistre.php">
<input type="text" name="nom">&nbsp;Entrez votre
nom<br>
<input type="submit" name="button" value="Enregistrer
la variable">
</form>
</body>
</html>
```

Créer la page nommée enregistre.php, et insérez le code ci dessous:

```
<?
//on démarre la session
session_start();
//on enregistre la variable nom dans la session ouverte
session_register("nom");
// $_SESSION['nom'] = $nom;
header('location: affiche.php');

?>
```

Création de la page affiche.php pour visualiser le résultat

```
<?
//on démarre la session
session_start();

//on affiche la variable enregistrée précédemment
print 'votre nom est '.$HTTP_SESSION_VARS[nom];
//vous pouvez l'afficher sous cette forme avec les
nouvelle version de php
print '<br>votre nom est '.$_SESSION[nom];
// liens page vérifier nommée verif.php
print '<br><a href="verif.php">Vérifier
l'enregistrement</a>;
```

```
?>
Explication:
```

Vous pourrez remarquer que lors de l'enregistrement de la variable "nom" en session, le signe \$ n'est pas ajouter dans session_register, si vous l'ajoutez, la variable ne sera pas enregistrée.

Avec les nouvelles version de PHP, vous pourrez rencontrer un problème en récupérant la variable nom simplement en utilisant \$nom, vous devrez la récupérer de cette manière \$_POST[nom] ou \$_HTTP_POST_VARS[nom]

Vérifier que la variable est enregistrée (page verif.php)

```
<?
session_start();

if( session_is_registered("nom") ) {

// la variable de session nom est enregistrée
print 'la variable nom est enregistrée , votre nom est '.
$_SESSION[nom];

//lien de suppression de variable et destruction de
session
print '<a href="out.php">Quitter la session</a>;
}
else{
// la variable n'est pas enregistrée, on renvoi sur le
formulaire

print '<a href="form.php">Cliquez ici pour vous
enregistrer</a>;

}

?>
```

Page de destruction de session et variable (out.php)

```
<?
session_start();

//suppression de la variable de session
session_unregister("nom");

//par mesure de sécurité, on détruit toutes les variables
ainsi que la session en cours
session_unset();
session_destroy();

// la variable nom n'est plus enregistrée, vous pouvez
vérifier avec les liens ci-dessous
print '<br><a href="verif.php">Vérifier
l'enregistrement</a>;
print '<br><a href="affiche.php">Afficher la
variable</a>;
```

```
?>
En cliquant sur les liens, vous ne devrez plus voir le
résultat de la variable nom, car celle ci a été détruite,
n'oubliez pas de faire cette manipulation, car une
variable enregistrée reste valide jusqu'à l'expiration de
la session.
```

Vous avez maintenant toutes les cartes en mains pour réaliser vos scripts et manipuler les sessions.

! ATTENTION !

la fonction session_start(); doit être lancée avant tout envoi de code html, print echo ou autre affichage, si vous ne respecter pas cette condition, vous aurez des erreurs de HEADERS que l'on voit souvent ;-)